Guide QC and analysis pipeline

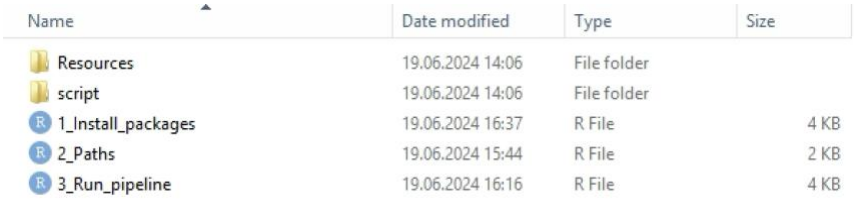
@Kira Hoeffler

Here is an overview of the actions needed before starting the QC and analyses and tips on installation.

Important steps are highlighted in red.

Running the pipeline

1. Download the pipeline from Github and unzip it. Download the Resources folder from the link on Github. Save both at the location you intend to use as your working directory.
    - **Important**: Ensure that the 1_Install_packages, 2_Paths, 3_Run_pipeline, and the script folder are in the working directory.



Name	Date modified	Type	Size
Resources	19.06.2024 14:06	File folder	
script	19.06.2024 14:06	File folder	
1_Install_packages	19.06.2024 16:37	R File	4 KB
2_Paths	19.06.2024 15:44	R File	2 KB
3_Run_pipeline	19.06.2024 16:16	R File	4 KB

2. **Open script 1_install_packages.R**
    a. Update the **array.type** to one of "EPICv1", "EPICv2" or "450K"
    b. Run the code lines. If you experience **any issues with the installation,** please consult the installation guide further down in this document.
3. Please **save your samplesheet in an Excel file, with the following columns:**
    - AMP_Plate
    - Sentrix_ID
    - Sentrix_Position
    - Sex – coded as "M" or "F"
    - Age (numeric)
    - Ancestry (genetic ancestry that can be self-reported or predicted). If Ancestry is unknown, please change it to "unknown".
    - Case_Control – coded as "Case" or "Ctrl"

    Complete data is expected. **It is crucial to ensure that the samplesheet is accurately coded with the correct column names and proper coding for Sex and Case_Control to run the pipeline.**

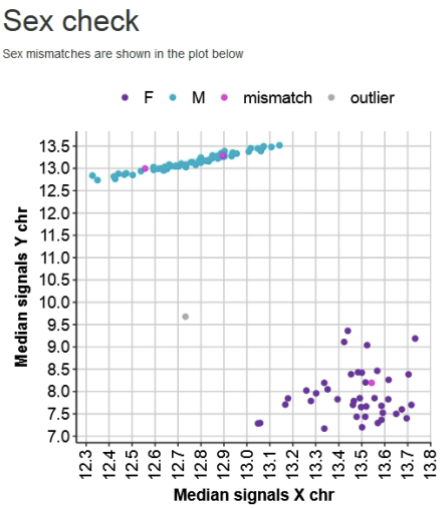
    We will generate a column called Basename using "SentrixID_SentrixPosition" for the sample names if this column is not already present in your sample sheet.

4. **Update the information in 2_Paths.** You do not need to run the 2_Paths.R script, just save your changes.
    a. Optional step: Use this only if you have expert knowledge:
        i. We provide the option to define additional samples in this script that you want to exclude, e.g., if there is a known genotyping mismatch.

ii. Add the sample's Basename (Sentrix_ID_Sentrix_Position") to the additional_outliers vector and a reason for exclusion, e.g., "genotyping_mismatch," to the reason_exclusion vector."

5. **Open the 3_Run_pipeline.R script** to see an overview of the scripts and the order in which they should be run.
   - The pipeline is divided into four parts (plus the setup), and an HTML report is generated after each part, providing an overview of the plots generated.
   - Please pay attention to warning messages when running the pipeline!

   a. Adapt the path to the 2_Paths.R script in the source() function at the start of the script.
   b. The code chunk "SET PATHS" must be run every time you return to running the pipeline after closing it.
   c. Run the first part of the pipeline under the header "PART ONE – SET-UP".
   d. Run the second part of the pipeline under the header "PART TWO – QC and Filtering pipeline I"
      i. **Important:** Check if clear outliers have been correctly identified ("grey dots") in the plot in the generated "Report1_QC_Sex.html" in your working directory.
      ii. <u>If the outliers have not been correctly identified:</u>
         1. Identify which X and Y coordinates in the sex check plot can be used to separate the outliers from the main female and male clusters. A larger version of the plot can be found in output/Figures/Sex_plot_mismatch. Here is an example of how it looks when the outlier in grey is correctly identified:

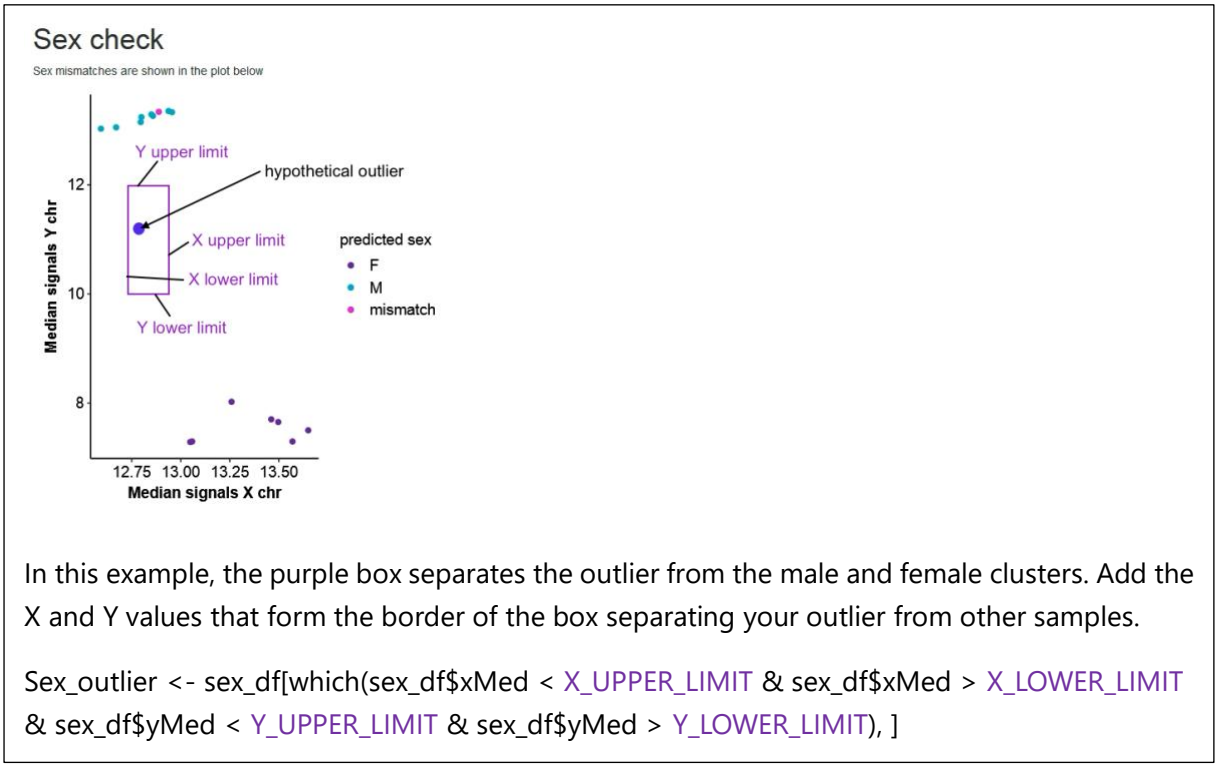


         2. if the default axes coordinates of x < 10.6 and y < 10 do not separate your outliers from the male and female clusters, please open the "2_Sex_check" script in the script folder and adapt the following line in the middle of the script (example in the box below):

```
# SEX OUTLIER (TRESHOLDS SET BASED ON PLOT)
sex_outlier <- sex_df[which(sex_df$xMed < 10.6 & sex_df$xMed > 0 & sex_df$yMed < 10 & sex_df$yMed > 0), ]
```

Save your changes and return to the main "3_Run_pipeline" script. Re-run the code to run "2_Sex_check.R" and generate the report:

```
source("script/2_Sex_check.R")

# CREATE A REPORT
source("2_Paths.R")
rmarkdown::render("script/Report_Part1.Rmd",
                  output_file = paste0(dir_gen,"Report1_QC_Sex.html"),
                  params = list(dir_gen = dir_gen))
```



In this example, the purple box separates the outlier from the male and female clusters. Add the X and Y values that form the border of the box separating your outlier from other samples.

Sex_outlier <- sex_df[which(sex_df$xMed < X_UPPER_LIMIT & sex_df$xMed > X_LOWER_LIMIT & sex_df$yMed < Y_UPPER_LIMIT & sex_df$yMed > Y_LOWER_LIMIT),]

e. You can run Part 3 of the pipeline, "PART THREE – QC and Filtering Pipeline II," once the sex outliers have been correctly identified.
   i. **Important:** Check if all obvious outliers have been removed in the last density plot (with the grey background) in "Report2_QC.html" in the working directory. If that is not the case:
      1. Similar to the sex outlier example, select X and Y coordinates that separate the outlier from the other samples.
      2. Open the script "4_Filtering_autosomes" in the "script" folder.
      3. Go to the outlier section at the end of the script:

```
### OUTLIERS ###

# if there is an outlier in the filtered beta distribution plot, select an X value,
# and Y values that separate the outlier from the other samples. Run the code below to identify the outlier:

# RUN THIS CODE (REMOVE THE # AT THE START OF THE NEXT TWO LINES)
#outlier <- dens_df_long$Sample[which(dens_df_long$X == x_value & dens_df_long$value <= y_upper_limit & dens_df_long$value >= y_lower_limit)]
#outlier
```

      4. Add values for x_value, y_upper_limit, and y_lower_limit that separate the outlier(s) from the other samples in this code and remove the hashtags.

```
outlier <- dens_df_long$Sample[which(dens_df_long$X == x_value & dens_df_long$value <= y_upper_limit & dens_df_long$value >= y_lower_limit)]
print(outlier)
```

      5. Re-run the respective script. The outliers will be printed at the end.
      6. Add the names of the outliers to the outlier vector at the start of the respective script, e.g., outlier <- c("sample_name1", "sample_name2"):

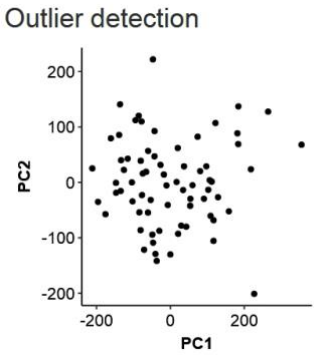
```
### OUTLIER SELECTION (ADD OUTLIER WHEN THERE IS STILL AN OUTLIER AFTER RUNNING THE SCRIPT FOR THE FIRST TIME, SEE GUIDELINES) ###

outlier <- c() #identify outlier at the end of the script
```

      7. Re-run the script once more.

8. Re-run the report generation via the "3_Run_pipeline.R" script.

```
# CREATE A REPORT
source("2_Paths.R")
rmarkdown::render("script/Report_Part2.Rmd",
                  output_file = paste0(dir_gen, "Report2_QC.html"),
                  params = list(dir_gen = dir_gen))
```

f. You can run the fourth part of your pipeline, "PART FOUR – Normalization, Imputation" if all obvious outliers have been removed in "Report_Part3_QC.html".
   i. **Important:** Check if there are any remaining obvious outliers in the outlier detection PCA plot in "Report3_Normalisation_Imputation_PCA.html.



<u>In the unlikely case that there are outliers:</u>

1. Open the "5_Normalisation_imputation_autosomes" script in the script folder and go to the end of the script.
2. Adapt the following lines by specifying which PC (PC1 or PC2) separates the outlier from the other samples and the value of this PC that separates the outlier from the other samples. Remove the hashtags.

```
# ADAPT AND RUN THIS CODE (REMOVE THE # AT THE START OF THE NEXT LINE), CHANGE WHICH PC SEPARATES THE OUTLIER AFTER THE $ SYMBOL AND THE NUMBER
outlier <- rownames(pca_scores[which(pca_scores$PC2 > 200), ])
print(outlier)
```

3. Re-run the entire script. It should print the outlier sample names at the end.
4. Open the "4_Filtering_autosomes" script in the script folder and add the outlier samples to the outlier vector at the start of the script, e.g:

```
### OUTLIER SELECTION (ADD OUTLIER WHEN THERE IS STILL AN OUTLIER AFTER RUNNING THE SCRIPT FOR THE FIRST TIME, SEE GUIDELINES) ###

outlier <- c("9741779074_R04C02", "9904796179_R05C02") # identify outlier at the end of the script
```

5. Save the script.
6. Re-run PART THREE of the pipeline from source("script/4_Filtering_autosomes.R") and then the entire PART FOUR.

```
source("script/4_Filtering_autosomes.R")

# CREATE A REPORT
source("2_Paths.R")
rmarkdown::render("script/Report_Part2.Rmd",
                  output_file = paste0(dir_gen, "Report2_QC.html"),
                  params = list(dir_gen = dir_gen))
```

```
#########################################################################################################################
# PART FOUR - Normalization, imputation
#########################################################################################################################

# RUN THE PIPELINE SCRIPTS 5-6
source("script/5_Normalisation_Imputation_autosomes.R")
source("script/6_Ancestry.R")

# CREATE A REPORT
source("2_Paths.R")
rmarkdown::render("script/Report_Part3.Rmd",
                  output_file = paste0(dir_gen, "Report3_Normalisation_Imputation_PCA.html"),
                  params = list(dir_gen = dir_gen)
```

ii. Check if there are any obvious batch effects in colored PCA plots (e.g. cases and controls cluster with AMP plate clusters → indicates that they have been run in two different batches. See review article on possible solutions.
iii. If your cohort consists of more than one genetic ancestry group, check ancestry prediction plot at the bottom of Report3 to see if the clusters match the Ancestry in your sample sheet.

g. You can run the fifth part of the pipeline, "PART FIVE – Analysis," when there are no remaining outliers in the outlier detection PCA plot in "Report3_Normalisation_Imputation_PCA.html."
    i. Choose the model with lambda value close to 1 and no obvious inflation.
    ii. Refer to the review if all your models are inflated.
        - VIF values above 3-5 indicate multicollinearity issues. Control probe PCs with VIF > 5 are automatically excluded from the analysis.

Installation guide

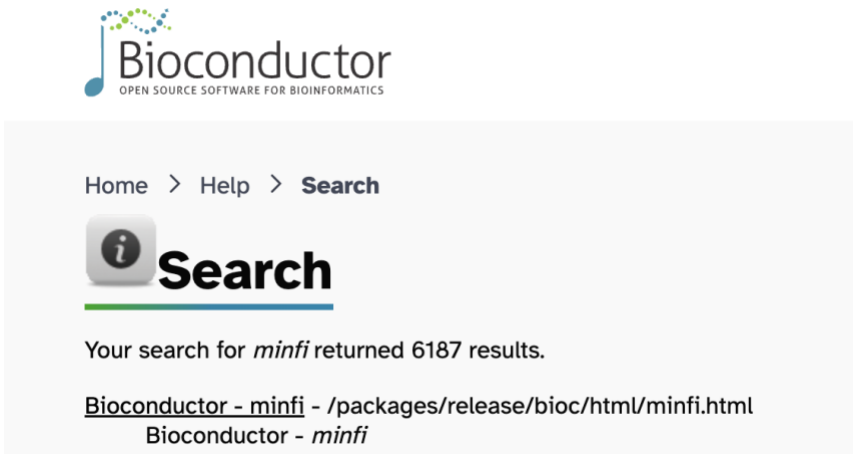
Two reasons why the **installation of new packages might not work** for you are:

1. The functions for installing from Bioconductor or GitHub in the script will not work if you do **not have an internet connection** on your secure server. In that case, please install the packages manually (see below).
2. You will encounter errors when it is **not possible to automatically install some of the dependencies** on your system. This means that the package you are trying to install first requires its dependencies to be installed. The error message will indicate which dependencies are missing. Please install these packages manually (see below) if the standard functions like install.packages() and BiocManager::install() do not work for you.

Install manually from Bioconductor

1. Search for the package name on the Bioconductor website: [Bioconductor - Home]

2. Select the first hit



3. Download the source package (at the bottom of the website):

4. Upload the package folder to your server
5. In Rstudio,
   a. Define your package path:
   package_path <- "Path_to_folder"

b. Install the package:
install.packages(package_path, repos = NULL, type = “source”)

Install manually from Github

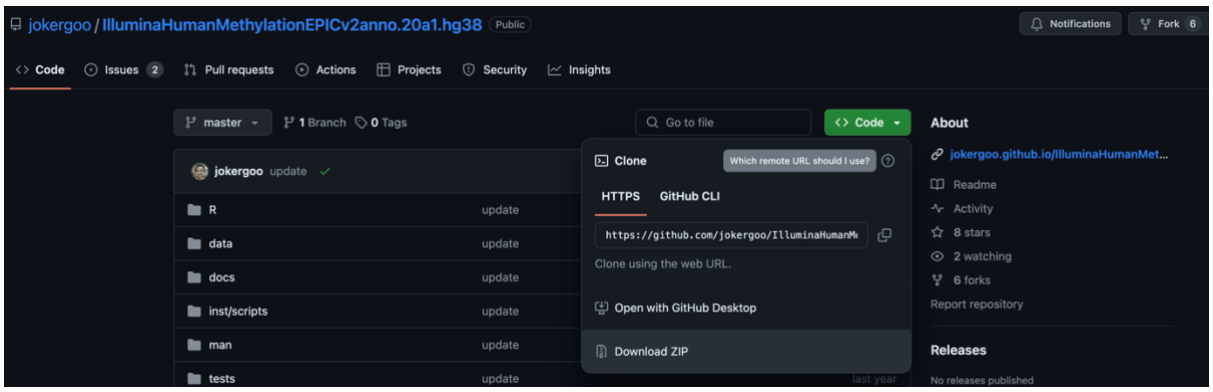
You will need the following three packages for the EPICv2 pipeline:

GitHub - schalkwyk/wateRmelon: Illumina 450 and EPIC methylation array normalization and metrics

GitHub - jokergoo/IlluminaHumanMethylationEPICv2manifest

GitHub - jokergoo/IlluminaHumanMethylationEPICv2anno.20a1.hg38

1. Open the link to the Github package (see above)
2. Go to **Code** and select **download ZIP** to download the package:



3. Upload the zip folder to your server
4. Unpack the zip folder / extract
5. In Rstudio,
   a. Define your package path:
   package_path <- “Path_to_folder”
   b. Install the package:
   install.packages(package_path, repos = NULL, type = “source”)